

Assignment 2: Coding Basics

Andi Scroggs

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.
part1 <- seq(1, 55, 5)
# I created a sequence of numbers one through 55 by an increment of five and named it "part1".
# seq(from, to, by)
```

```
#2.
mean(part1)
```

```
## [1] 26
```

```
# find the mean of the sequence of numbers I just created
median(part1)
```

```
## [1] 26
```

```
# find the median of the sequence of numbers I created

#3.
mean(part1)>median(part1)
```

```
## [1] FALSE
```

```
# this is asking whether the mean of the sequence of number is greater than the median of the sequence  
# the result is FALSE, thus the mean is not greater than the median of the sequence.  
mean(part1)==median(part1)
```

```
## [1] TRUE
```

```
# this is asking whether the mean is equal to the median; the result is TRUE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
# creating three separate vectors  
# vector for student names  
student_names <- c("Beth", "Steve", "Carl", "Alex")  
# vector for the students test scores  
test_scores <- c(95, 80, 86, 91)  
# vector indicating if the students were on scholarship  
scholarship <- c(TRUE, FALSE, TRUE, TRUE)  
# TRUE is they are on scholarship; FALSE is they are not on scholarship  
  
#creating a students data.frame with three columns and four rows  
students_dataframe <- data.frame(student_names, test_scores, scholarship)  
  
#setting the column names in the data.frame  
names(students_dataframe) <- c("Student Name", "Test Score", "Scholarship (T or F)")
```

9. QUESTION: How is this data frame different from a matrix?

Answer: A data frame is a data structure where the columns can have different types of data, however, within the columns themselves, the data type must be the same. For example, there can be columns with numerical data, logical data, or character data within a data frame. In a matrix there can only be one type of data throughout the whole data structure. The columns and rows must be the same type of data (e.g. only numerical or logical or characters).

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input

13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
# I created a new function - "fun10" - my function has one input x (look at "reproducibility_codingbasics" for help)
# if x is greater than 50 it will say "Pass"
# if x is less than 50 it will say "Fail"
fun10 <- function(x) {
  if(x > 50) {
    "Pass"
  }
  else {
    "Fail"
  }
}

#11. Create a function using ifelse()
# (look at "reproducibility_codingbasics" for help)
# ifelse (function, if True, if False)
fun11 <- function(x) {
  ifelse(x > 50, "Pass", "Fail")
}

#12a. Run the first function (fun10) with the value 52.5

fun12 <- fun10(52.5) # output is "Pass"

#12b. Run the second function (fun11) with the value 52.5

fun13 <- fun11(52.5) # output is "Pass"

#13a. Run the first function with the vector of test scores

# fun13a <- fun10(test_scores)

#13b. Run the second function with the vector of test scores

fun13b <- fun11(test_scores)
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The `ifelse` function worked and the `if...else` function did not work. This is because the `ifelse` function is vectorized while the `if...else` function is not. The `ifelse` function is able to compute the whole vector at once, while the `if...else` function will compute the values in the vector individually.

NOTE Before knitting, you'll need to comment out the call to the function in Q13 that does not work. (A document can't knit if the code it contains causes an error!)